

Guía para el equipo — Proyecto YPF / Vaca Muerta

Esta guía es para que cualquiera del equipo pueda bajar el proyecto a su computadora, entender qué hace cada parte, y empezar a trabajar sin pisar el trabajo de los demás. Está pensada en nivel junior: si algo te parece "obvio", mejor, significa que está bien explicado.

Parte 1 — Cómo clonar el proyecto en tu PC

"Clonar" es bajar una copia completa del repositorio (código + historial de cambios) a tu computadora.

Paso 1: Tené Git instalado

Abrí una terminal y escribí:

```
git --version
```

Si te devuelve un número de versión, ya lo tenés. Si no, instalalo desde git-scm.com.

Paso 2: Cloná el repositorio

En la terminal, parate en la carpeta donde querés que quede el proyecto (por ejemplo Documentos/) y corré:

```
git clone https://github.com/cristian-aguilera-ing/proyecto-ds-ypf.git
```

Esto crea una carpeta `proyecto-ds-ypf/` con todo el código adentro.

Paso 3: Entrá a la carpeta y armá tu entorno virtual

El proyecto usa un entorno virtual de Python para no mezclar las librerías del proyecto con las que ya tengas instaladas en tu sistema.

```
cd proyecto-ds-ypf
python3 -m venv venv
```

Actívalo:

```
# Mac / Linux
source venv/bin/activate

# Windows
venv\Scripts\activate
```

Vas a ver que el prompt de la terminal cambia y aparece `(venv)` al principio. Eso significa que está activado.

Paso 4: Instalá las dependencias

```
pip install -r requirements.txt
```

Esto instala automáticamente todas las librerías que el proyecto necesita (pandas, matplotlib, xgboost, lightgbm, etc.), ya con las versiones correctas.

Paso 5: Abrí el proyecto en tu editor

Si usás VS Code:

```
code .
```

Y listo, ya tenés todo el proyecto funcionando local.

Parte 2 — Cómo trabajar con ramas (para no romper `main`)

¿Por qué no laburamos todos directo sobre `main`?

`main` es la rama "oficial" del proyecto — la que tiene que quedar siempre funcionando. Si todos commiteamos directo ahí, cualquier error de uno rompe el trabajo de todos, y además no queda claro quién hizo qué ni cuándo.

La solución: **cada uno trabaja en su propia rama**, y recién cuando esa parte está lista y probada, se junta (merge) con `main`.

Paso 1: Asegurate de estar actualizado antes de arrancar

Siempre, antes de crear una rama nueva, traé los últimos cambios de `main`:

```
git checkout main
git pull origin main
```

Paso 2: Creá tu propia rama

Nombrá la rama según lo que vas a hacer, por ejemplo `clustering-kmeans`, `notebook-04`, `fix-shap`, etc.

```
git checkout -b clustering-kmeans
```

Esto crea la rama y te para automáticamente dentro de ella (fijate que el nombre de la rama cambia en la terminal o en VS Code abajo a la izquierda).

Paso 3: Trabajá normalmente

Editá archivos, corré notebooks, lo que necesites. Cuando quieras guardar un avance:

```
git add .
git commit -m "Agrego primer intento de clustering con K-Means"
```

Podés hacer todos los commits que quieras mientras trabajás — no hace falta que cada commit sea "perfecto".

Paso 4: Subí tu rama a GitHub

```
git push origin clustering-kmeans
```

La primera vez que hacés push de una rama nueva, Git te va a tirar el comando exacto a usar si te olvidaste el nombre — no te preocupes si te marca error, simplemente copió y pegá lo que te sugiere.

Paso 5: Pedí que tu trabajo se junte con `main` (Pull Request)

Andá a GitHub, a la página del repositorio, y vas a ver un botón tipo "Compare & pull request" (aparece solo cuando subiste una rama nueva). Ahí:

1. Escribís un título corto de qué hiciste.
2. En la descripción, contás brevemente qué cambiaste.
3. Creás el Pull Request (PR).

Esto **no junta nada todavía** — es una propuesta de cambio que el resto del equipo puede revisar antes de aceptarla.

Paso 6: Revisión y merge

Alguien del equipo (o vos mismo si ya está charlado) revisa el PR y, si está todo bien, le da a "**Merge pull request**". Recién ahí tu trabajo pasa a formar parte de `main`.

Regla de oro del equipo

Nunca trabajes directo sobre `main`. Siempre: `git pull` de `main` → rama nueva → trabajar → push → Pull Request → merge.

Si en algún momento tenés dudas de en qué rama estás parado, corré:

```
git branch
```

La rama con el `*` adelante es en la que estás.

Parte 3 — El proyecto en sí: de qué se trata y a dónde llegamos

3.1 La idea en una frase

Queremos **predecir cuánto petróleo y gas va a producir un pozo no convencional (shale/tight) de Vaca Muerta el mes que viene**, usando datos históricos de ese mismo pozo y de pozos parecidos. Además, en una segunda etapa (todavía no arrancada), vamos a **agrupar pozos en "familias" según cómo es su curva de producción** (clustering), para entender mejor los distintos comportamientos que hay en la cuenca.

Es un proyecto de **regresión supervisada** (para la predicción) + **clustering no supervisado** (para el agrupamiento).

3.2 El dataset: qué tenemos y de dónde sale

Los datos son públicos, de la **Secretaría de Energía de la Nación Argentina** (datos.gob.ar). El archivo principal se llama "*Producción de Pozos de Gas y Petróleo No Convencional*" y ya viene filtrado: solo tiene pozos shale/tight, no hace falta separarlos nosotros de los convencionales.

Algunos números para tener la foto completa:

Qué

Cuánto

Filas	421.046
Columnas	40 (ahora 39, después de limpieza)
Rango de fechas	2006 a 2026
Una fila =	un pozo, en un mes y año puntual

O sea: cada fila es "el pozo X, en el mes Y del año Z, produjo tanto petróleo y tanto gas". Como cada pozo aparece una vez por mes a lo largo de toda su vida útil, el mismo pozo puede tener cientos de filas (una por cada mes que estuvo activo).

Columnas clave que vas a ver seguido en el código:

- `prod_pet / prod_gas` : producción de petróleo y gas de ese pozo ese mes. `prod_pet` es la variable que estamos tratando de predecir.
- `profundidad` : profundidad del pozo.
- `formacion` : formación geológica donde está el pozo (ej. "Vaca Muerta", "Lajas").
- `cuenca` : cuenca (zona geográfica) donde está.
- `idpozo` : identificador único de cada pozo.

3.3 Estructura de carpetas del repositorio

```

proyecto-ds-ypf/
├── data/
│   ├── raw/           ← los CSV originales, tal cual los descargamos, nunca se tocan
│   └── processed/      ← versiones ya limpias/transformadas, generadas por nuestro código
├── notebooks/         ← acá vive el análisis, paso a paso, explicado con texto y gráficas
├── src/                ← el código "serio": funciones reutilizables que los notebooks
├── api/                ← (a futuro) la API que va a exponer el modelo para hacer predicciones
├── models/            ← los modelos ya entrenados, guardados en disco (.joblib)
├── reports/           ← gráficos e informes para mostrar resultados
├── requirements.txt    ← la lista de librerías que hay que instalar (ver Parte 1)
├── Dockerfile          ← (a futuro) para poder correr todo el proyecto empaquetado
└── README.md          ← presentación general del proyecto

```

Idea clave de por qué está separado así: `data/raw` nunca se edita a mano (si rompemos algo, siempre podemos volver a partir de ahí). `src/` tiene el código que se reutiliza en varios lados, para no copiar y pegar la misma función en cada notebook. `notebooks/` es donde "contamos la historia" del análisis: qué hicimos, por qué, y qué encontramos.

3.4 Qué hace cada archivo de `src/` (hasta ahora)

`src/data_loader.py` — Se encarga solo de **leer y validar** los datos crudos, sin cambiar nada en ellos:

- `cargar_produccion_no_convencional()` : lee el CSV principal.
- `cargar_capitulo_iv_pozos()` : lee un CSV complementario con metadatos de pozos.
- `validar_calidad()` : no modifica nada, solo revisa y reporta problemas (nulos, duplicados, valores raros).

`src/preprocessor.py` — Se encarga de **limpiar y transformar** los datos (a diferencia del

`src/preprocessor.py` — Se encarga de limpiar y transformar los datos (a diferencia del anterior, que solo lee). Tiene funciones como:

- Sacar columnas casi vacías (`vida_util`, `observaciones`, que tenían más del 90% de nulos).
- Corregir la `profundidad` cuando tiene valores imposibles (se completa con la mediana de esa formación geológica).
- Poner en 0 los pocos valores de producción negativa que no tienen sentido físico.
- Marcar qué filas pertenecen al 2026 (año todavía incompleto en el dataset), para poder excluirlas de los análisis de tendencia sin perder esas filas para el modelo.

3.5 Qué hace cada notebook (hasta donde llegamos)

`01_eda_limpieza.ipynb` — El primer análisis exploratorio (EDA = Exploratory Data Analysis).
Acá:

- Vemos cuántos nulos hay por columna, con gráficos.
- Encontramos que el valor rarísimo de `profundidad` (378.939, imposible para un pozo real) era en realidad un solo pozo con un error de carga, repetido en todos sus registros — no un montón de outliers sueltos.
- Confirmamos visualmente que la producción está muy sesgada (muchos pozos producen poco, pocos producen mucho), lo cual es normal en este tipo de datos.
- Vemos que el 2026 muestra una "caída" de producción que en realidad es solo que todavía no están cargados los 12 meses del año.

`02_feature_engineering.ipynb` — Acá creamos variables nuevas a partir de las que ya teníamos, para darle al modelo más información útil. Por ejemplo:

- `antiguedad_meses`: hace cuántos meses que el pozo está produciendo.
- `prod_pet_mes_anterior`: cuánto produjo ese mismo pozo el mes anterior (resultó ser la variable más importante de todas).
- `declinacion_pet_anterior`: cuánto varió la producción entre el mes anterior y el anterior a ese (ojo con esta, ver el punto de "fuga de datos" más abajo).
- `ratio_gas_petroleo`: relación entre gas y petróleo producido.
- `formacion_cod` / `cuenca_cod`: versión numérica de las columnas de texto `formacion` y `cuenca` (los modelos no entienden texto, necesitan números).

`03_modelado_supervisado.ipynb` — Acá entrenamos los modelos que predicen `prod_pet`. Este notebook tiene varias "vueltas" (iteraciones), y vale la pena entender la lógica de cada una porque es parte importante de lo que vamos a defender:

1. **Modelo base** (XGBoost, 7 variables): anduvo bien en general, pero predecía mal los pozos con picos de producción muy altos.
2. **Un error que aprendimos a detectar — "fuga de datos" (data leakage)**: probamos agregar una variable (`declinacion_pet`, calculada con el mes *actual*) y el modelo mejoró demasiado de golpe (R^2 de 0.978). Eso prendió una alarma: cuando algo mejora "demasiado", hay que sospechar. Descubrimos que esa variable, sin querer, contenía escondida la respuesta que el modelo tenía que adivinar. La sacamos y usamos en cambio `declinacion_pet_anterior` (calculada solo con datos del pasado, sin trampa).
3. Probamos un par de ideas más (agregar el máximo histórico del pozo, entrenar sobre el

logaritmo de la producción) — ninguna mejoró las cosas de fondo, así que las descartamos.

Está bien descartar intentos que no funcionan: forma parte normal de probar cosas.

4. Como pedía la consigna, entrenamos **dos algoritmos y los comparamos**: XGBoost vs. LightGBM, con las mismas 8 variables finales. LightGBM ganó por un margen chico pero consistente, así que **ese quedó como modelo principal**.

Resultado final (LightGBM, 8 variables): RMSE 439.34, R^2 0.888 — es decir, el modelo explica casi el 89% de la variación en la producción de petróleo.

Limitación que sabemos que tenemos y que vamos a mencionar en la defensa: en pozos con picos de producción muy altos (posibles reestimulaciones o refracturas), el modelo todavía subestima el valor real. Probamos tres formas distintas de solucionarlo y ninguna funcionó del todo — lo cual sugiere que falta una variable que el dataset no tiene (por ejemplo, si se programó una intervención en el pozo). Se documenta como limitación conocida, no se esconde.

3.6 Qué sigue (para coordinar quién hace qué)

1. `04_modelado_no_supervisado.ipynb`: clustering (K-Means) para agrupar pozos según su curva de declinación.
 2. Opcional si llega el tiempo: optimización de hiperparámetros (Optuna) y explicabilidad del modelo (SHAP).
 3. Entrega final: API en FastAPI + Dockerfile + README completo.
-

Resumen para tener siempre a mano

- **Clonar:** `git clone` → crear venv → `pip install -r requirements.txt`.
- **Trabajar:** nunca directo en main → `git pull origin main` → `git checkout -b mi-rama` → trabajar → `git push origin mi-rama` → Pull Request en GitHub → merge.
- **El proyecto:** predecir cuánto petróleo va a producir un pozo el próximo mes, con datos